

Nama : Sabrina Zahra

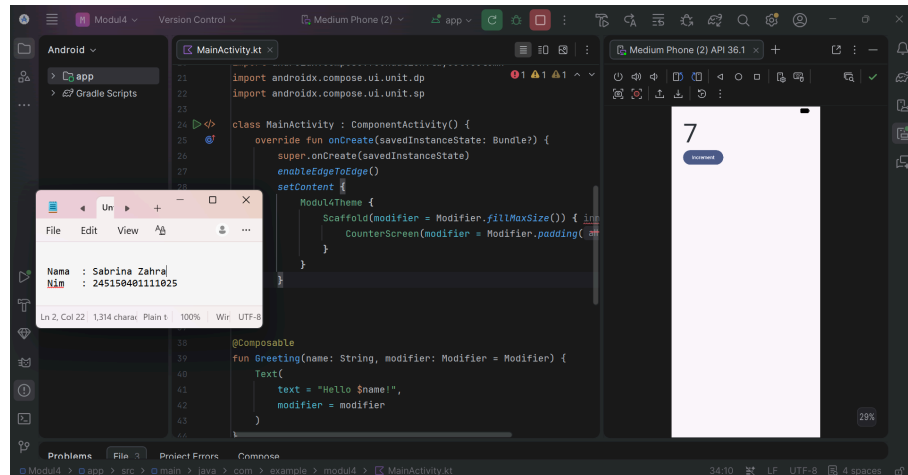
NIM : 245150401111025

Modul : Modul 4 (State Hoisting, Perubahan Konfigurasi, dan Side-effect)

Link github (modul & tiket) : <https://github.com/Sabrina150506/Modul4.git>

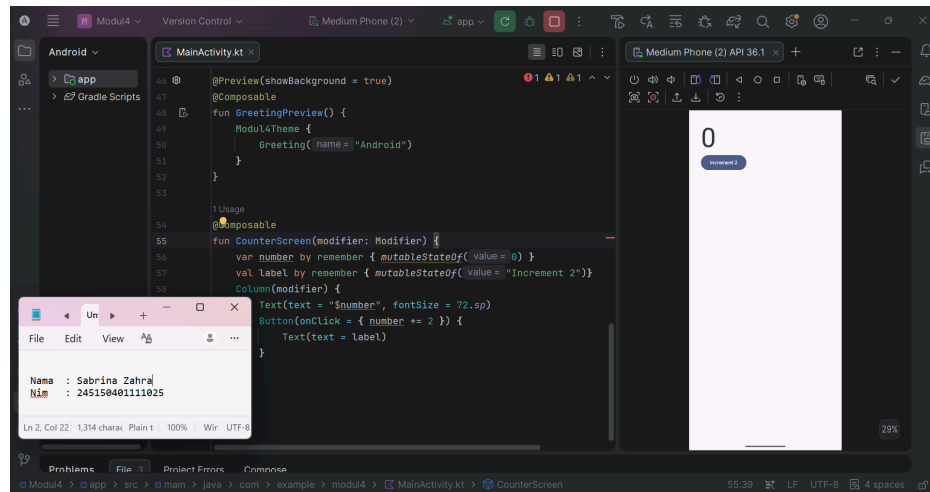
4.5.1 State Hoisting

1. Screenshot setelah tombol "Increment" diklik beberapa kali.

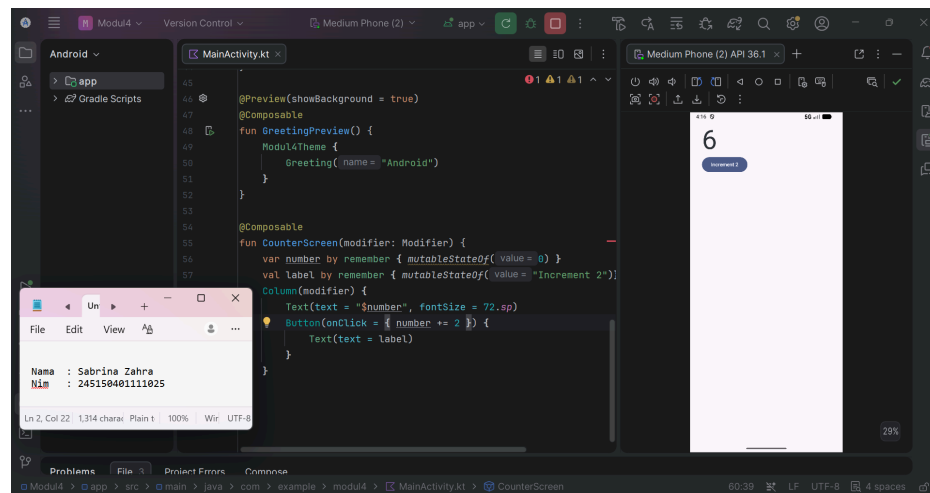


- Apa yang terjadi setiap kali tombol "Increment" diklik? setiap kali tombol Increment diklik, angka yang ditampilkan pada layar akan berubah dengan cara bertambah 1. Contohnya, dari 0 jika di klik akan menjadi 1, kemudian 2, 3, dan seterusnya.**
 - Apa yang menyebabkan hal tersebut terjadi? hal tersebut terjadi karena terdapat kode `number++` pada tombol `onClick` yang berfungsi untuk menambah nilai `number` sebanyak 1 ketika tombol diklik. Setelah nilainya berubah, Jetpack Compose akan memperbarui tampilan secara otomatis (recomposition) sehingga angka yang muncul di layar menjadi angka terbaru.**
 - Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut? variabel *state* yang berperan adalah `number`, hal tersebut ditunjukkan dengan adanya nilai `number` yang digunakan pada kode berikut `Text(text = "$number", fontSize = 72.sp)`**
 - Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut? variabel *state* yang berperan adalah `label`, karena digunakan untuk menampilkan tulisan pada Button, hal tersebut ditunjukkan pada kode berikut `Text(text = "$label")`**
2. Perubahan dimana nilainya akan ditambah 2 jika di klik dan nama tombolnya berubah jadi "increment 2"
- Perubahan apa yang perlu dilakukan dan dilakukan di mana? ada 2 perubahan yang perlu dilakukan pada fungsi `CounterScreen()` . Berikut detailnya:**
 - Perubahan pertama dilakukan pada kode `Button(onClick = { number++ })` yang diubah menjadi `Button(onClick = { number += 2 })`
 - Perubahan kedua dilakukan pada kode `val label by remember { mutableStateOf("Increment") }` yang diubah menjadi `val label by remember { mutableStateOf("Increment 2") }`

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

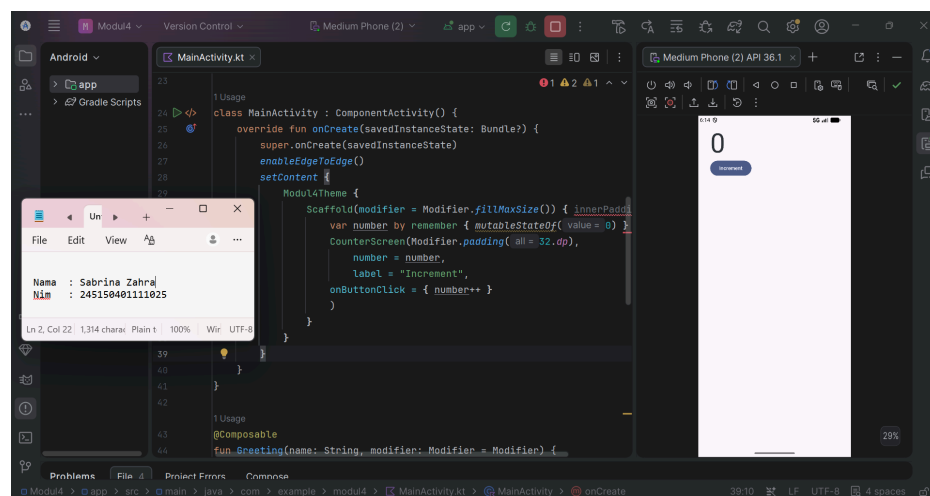


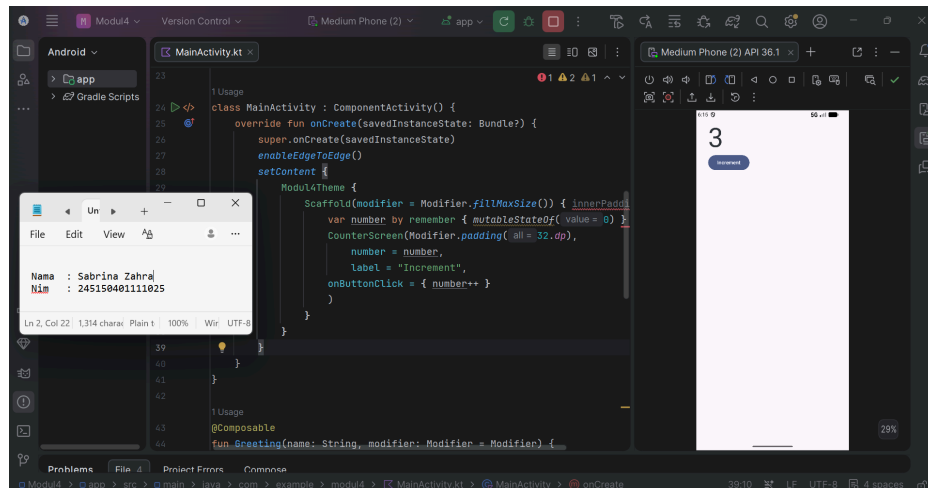
(sebelum tombol increment diklik)



(sesudah tombol increment diklik)

3. Perubahan kode dimana state number dideklarasikan di dalam fungsi onCreate()

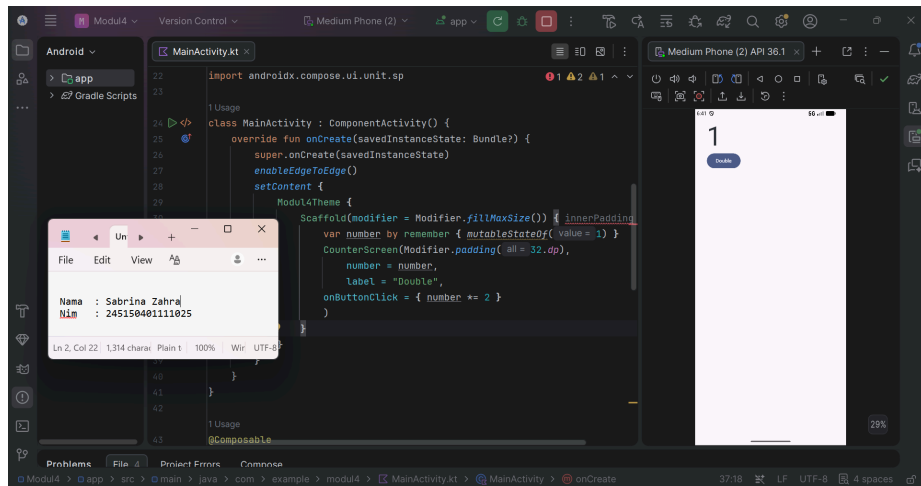




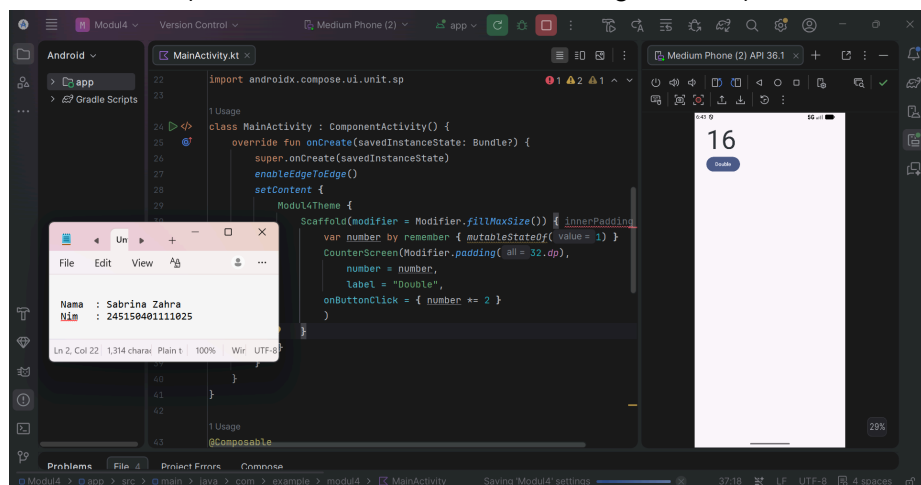
- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya? iya benar program tetap berjalan dengan baik dan dapat menampilkan hasil yang sama persis, selain itu setiap tombol Increment di klik, angka tetap bertambah 1.
 - b. Di bagian kode program manakah saat ini proses increment dilakukan? saat ini proses increment dilakukan dibagian lambda onClick ketika CounterScreen() dipanggil, berikut kode programnya `onClick = { number++ }`
 - c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan? state number yang digunakan oleh Composable Text dan Button "Increment" dideklarasikan di dalam `onCreate()`. Sedangkan CounterScreen() hanya menerima nilai number dan label melalui parameter. Jadi, state dikelola dari `onCreate()`, sementara CounterScreen() hanya menampilkan dan menggunakan data tersebut
4. Perubahan kebutuhan fungsional aplikasi dimana nilai awal menjadi 1, label menjadi Double, dan operasi `number++` diganti menjadi operasi penggandaan.
- a. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan. state number dideklarasikan di dalam `onCreate()`. Kemudian number, label, dan `onClick` dikirim ke CounterScreen() melalui parameter. Saat tombol diklik, `number++` akan menambah nilai number sebesar 1. Berikut kodenya:

```
var number by remember { mutableStateOf(1) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onClick = { number *= 2 }
)
```

- b. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.



(sebelum nilai variabel number digandakan)



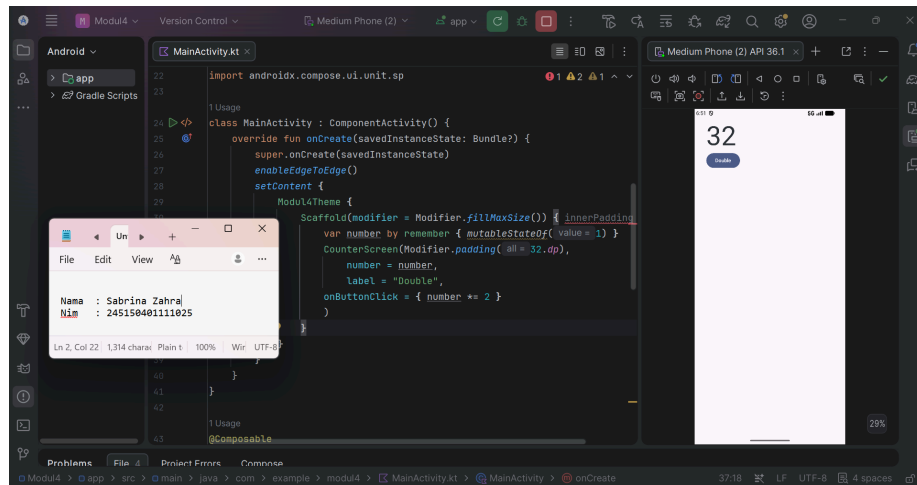
(sesudah nilai variabel number digandakan)

- c. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan? perubahan dilakukan pada fungsi onCreate(), khususnya pada deklarasi state dan pemanggilan CounterScreen().
 - d. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan. tidak ada perubahan pada logika CounterScreen() itu sendiri. Perubahan dilakukan pada nilai label, nilai awal number, dan lambda onClick di onCreate(). Hal ini menunjukkan manfaat state hoisting karena perubahan logika dapat dilakukan pada parent tanpa harus mengubah composable anak.
5. **Manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?** state hoisting membuat perubahan kebutuhan fungsional menjadi lebih mudah karena state dan logika program disimpan di composable induk, sedangkan CounterScreen() hanya bertugas menampilkan data dan menerima perubahan melalui parameter. Dengan cara ini, kode menjadi lebih rapi, mudah digunakan kembali, dan lebih mudah diuji. Jika ada perubahan pada logika program, perubahan tersebut dapat dilakukan tanpa harus mengubah bagian UI secara langsung. Hal ini juga menerapkan konsep Single

Source of Truth, yaitu data utama hanya dikelola di satu tempat, serta Separation of Concern, yaitu memisahkan pengelolaan data dan logika dari tampilan.

4.5.2 Perubahan Konfigurasi

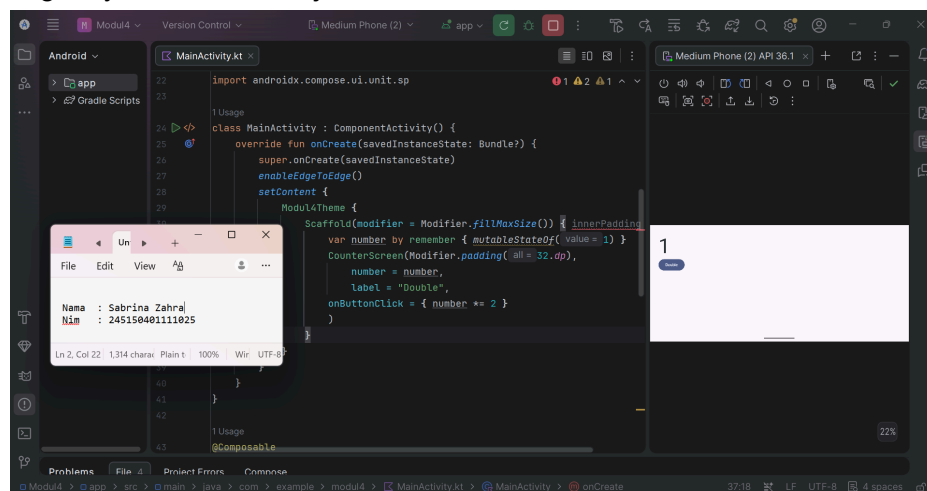
1. Screenshot tampilan antarmuka setelah tombol “Double” di klik beberapa kali



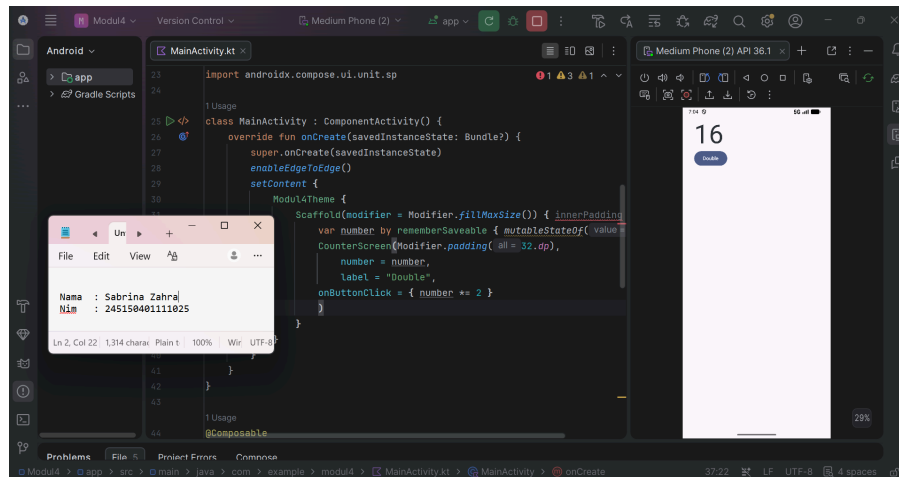
Catatan angkanya : 1, 2, 4, 8, 16, 32

2. Mengubah posisi perangkat menjadi landscape

- Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape. saat diubah ke posisi landscape angkanya berubah menjadi 1

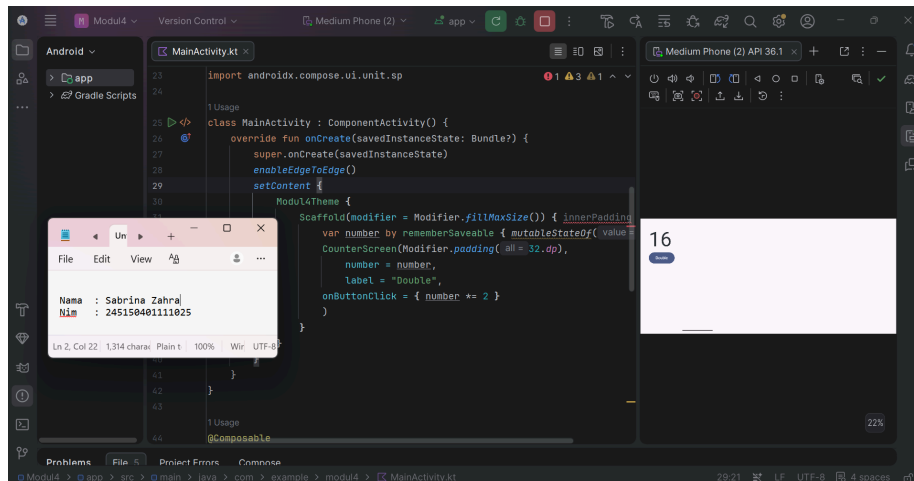


- Berapa angka yang ditampilkan? Mengapa terjadi demikian? angka yang ditampilkan adalah 1 hal tersebut terjadi karena remember hanya mempertahankan state selama proses recomposition. Ketika terjadi perubahan konfigurasi seperti rotasi layar, Activity dibuat ulang sehingga state yang disimpan menggunakan remember dapat hilang. Pada modul juga menjelaskan bahwa remember tidak mempertahankan nilai state ketika terjadi perubahan konfigurasi.
- Screenshot setelah ditambahkan kode `rememberSaveable`



3. Perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape setelah ditambahkan kode rememberSaveable

- Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape. angka yang ditampilkan adalah 1, 2, 4, 8, 16



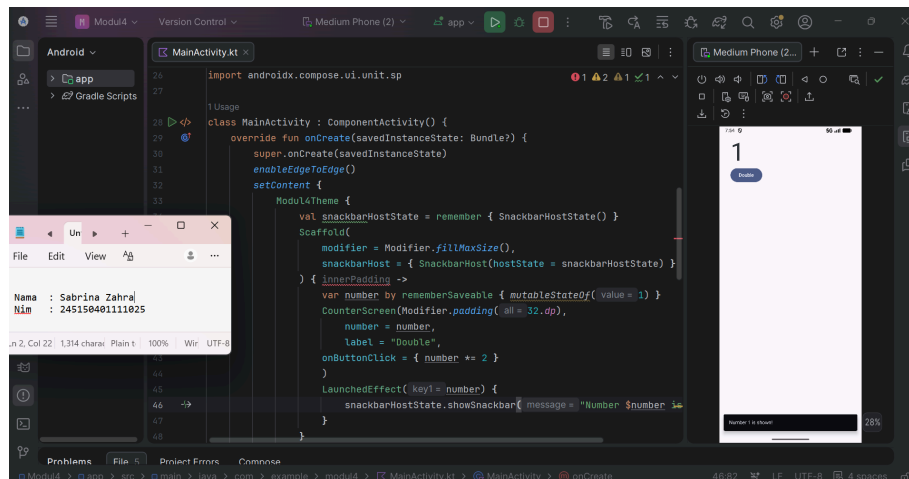
- Berapa angka yang ditampilkan? Mengapa terjadi demikian? angka yang ditampilkan tetap 8, hal tersebut terjadi karena rememberSaveable menyimpan state sehingga nilai tetap dapat dipulihkan ketika Activity dibuat ulang akibat perubahan konfigurasi.
 - Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember? rememberSaveable digunakan untuk menyimpan nilai state agar tetap ada ketika layar mengalami perubahan konfigurasi, seperti saat HP diputar dari portrait ke landscape. Sedangkan remember hanya menyimpan state selama Composable masih berjalan dan nilainya dapat hilang ketika Activity dibuat ulang. Jadi, perbedaannya adalah rememberSaveable dapat mempertahankan dan mengembalikan nilai state setelah Activity dibuat ulang, sedangkan remember tidak.
4. Beri contoh kasus nyata lain di mana penggunaan konstruk rememberSavable ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya! contoh penggunaan rememberSaveable dapat ditemukan pada form pendaftaran atau form login. Misalnya, pengguna sudah

mengisi nama, email, dan nomor telepon, lalu memutar layar HP. Dengan `rememberSaveable`, data yang sudah diisi akan tetap tersimpan dan ditampilkan sehingga pengguna tidak perlu mengisinya kembali. Hal ini membuat aplikasi lebih nyaman digunakan dan mencegah data yang sudah dimasukkan hilang saat terjadi perubahan konfigurasi.

4.5.3 LaunchedEffect

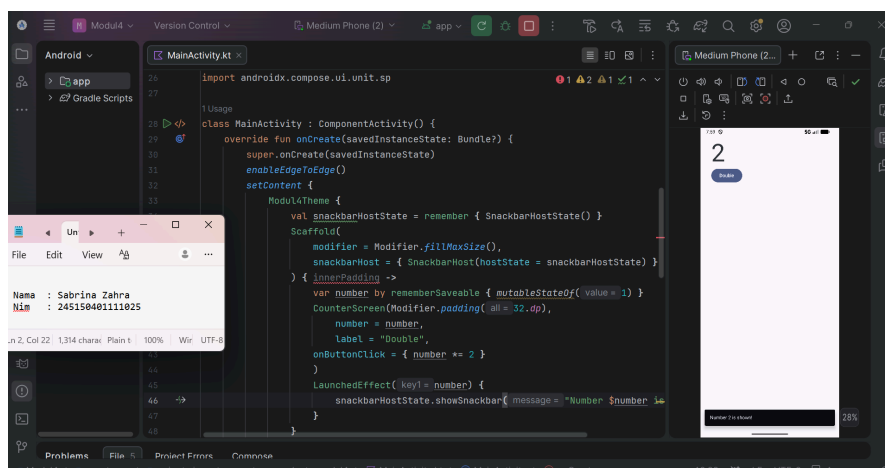
1. Setelah program dijalankan

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button “Double” apa yang ditampilkan di dalam layar? pada awal aplikasi dijalankan, akan muncul `SnackBar` dengan pesan `Number 1 is shown!`

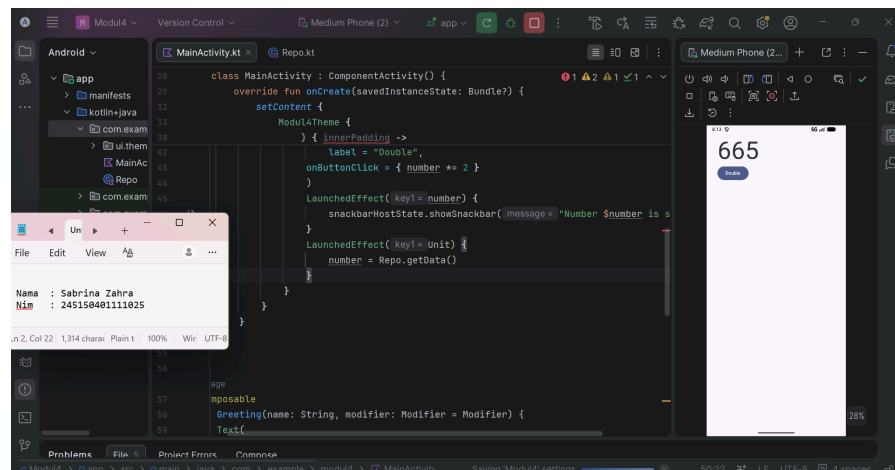


- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna? karena terdapat `LaunchedEffect(number)` yang langsung dijalankan ketika composable masuk ke dalam composition sehingga tidak diperlukan interaksi pengguna untuk menjalankannya. Oleh karena itu pesan tersebut bisa langsung muncul tanpa adanya interaksi.
- c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana? `SnackBar` dipicu oleh perubahan nilai `number`. Pada kode `LaunchedEffect(number)`, `number` digunakan sebagai key. Jadi, setiap kali nilai `number` berubah, `LaunchedEffect` akan dijalankan kembali dan menampilkan `SnackBar` dengan pesan “Number [nilai number] is shown!”.

2. Setelah dilakukan klik button “Double” satu kali



- a. **Apa yang terjadi?** ketika tombol Double diklik, nilai number akan menjadi dua kali lipat dari nilai sebelumnya. Setelah nilai berubah, Snackbar akan muncul dan menampilkan nilai number yang terbaru, misalnya "Number 2 is shown!".
 - b. **Mengapa hal tersebut terjadi? Jelaskan!** hal tersebut terjadi karena ketika tombol Double diklik, nilai number berubah. Nah perubahan tersebut menjadi pemicu `LaunchedEffect(number)` untuk menjalankan kembali coroutine dan menampilkan Snackbar dengan nilai terbaru. Hal tersebut sesuai dengan penjelasan modul mengenai `LaunchedEffect` yang akan menjalankan kembali blok ketika key yang digunakan berubah.
3. **Jalankan program setelah dilakukan perubahan dengan menambahkan class Kotlin baru bernama Repo**



- a. **Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?** saat aplikasi pertama kali dijalankan, nilai awal number yang ditampilkan adalah 1. Setelah itu, aplikasi akan mengambil angka acak melalui `Repo.getData()` lalu nilai number berubah menjadi angka acak antara 100 sampai 999.
 - b. **Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?** setelah sekitar 2 detik, nilai number berubah menjadi angka acak antara 100 sampai 999. Setelah angka berubah, Snackbar juga akan menampilkan nilai number yang terbaru.
 - c. **Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?** perubahan angka dipicu oleh `LaunchedEffect(Unit)` yang menjalankan `Repo.getData()`. Di dalam fungsi tersebut terdapat `delay(2000)`, sehingga aplikasi menunggu 2 detik sebelum menghasilkan angka acak. Setelah number berubah, `LaunchedEffect(number)` akan berjalan kembali dan menampilkan Snackbar dengan nilai terbaru.
4. **Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,**
- a. **Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?** `LaunchedEffect(number)` menggunakan number sebagai key, sehingga efek akan dijalankan kembali setiap kali nilai number berubah. Contohnya, ketika number berubah, Snackbar akan muncul lagi dengan nilai terbaru. Sedangkan `LaunchedEffect(Unit)` menggunakan Unit sebagai key, sehingga efek biasanya hanya dijalankan sekali saat Composable pertama kali ditampilkan dan tidak dijalankan kembali hanya karena state

berubah. Jadi, perbedaan utamanya terletak pada key yang digunakan dan kapan efek dijalankan kembali.

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

- `LaunchedEffect(number)` : dijalankan saat Composable pertama kali ditampilkan dan dijalankan kembali setiap kali number berubah.
- `LaunchedEffect(Unit)` : dijalankan saat Composable pertama kali ditampilkan dan cocok digunakan untuk proses yang cukup dilakukan sekali, seperti mengambil data awal.